CO1035: Game Concepts
Lab 2C: Field of Spikes (Game Prototype)

1. Introduction

The instructions in this lab sheet will guide you through creating (most of) a game called **"The Field of Spikes"**. The game is played by flying a plane and shooting at giant spiky spikes that have mysteriously appeared all over the world. This guide will walk you through building most of the game, and other features can be added in the coming weeks as they are introduced in the labs, or even better, you can try adding them even before then. Let's get started!!!

2. Setting Up

a. Create a new project named **Lab2C_FieldOfSpikes_Project** and open the project.

b. This project requires models that are not installed with the TL-Engine. Download the zipped folder **Lab2C_extramedia.zip** containing the models from Blackboard. It should contain a folder named "**media**" which contains the models you need. Copy the "**media**" folder and place it in your project's folder.

c. In the .cpp file of your project, change the default location for the project's media files using the code below so that it points to the "media" folder in your project's folder.

```
myEngine->AddMediaFolder(".\\media");
```

3. The Basic Game Scene

a. **Add code** to create a camera for your scene; **make it an FPS** camera for now so that you can look around the scene.

b. Use the "**floor.x**" mesh to **create** a floor model at the origin.

c. Use the "**skybox.x**" mesh to **create** a skybox at position (0, -960, 0).

d. Run your program to have a look at the scene you created.

e. Now use the "**plane.x**" mesh to create a model at the position (0, 10, 0). Then, in the game loop, **give the plane a forward speed** of 0.05 along the **z-axis** by adding the code

```
plane->MoveZ( 0.05 );
```

f. When you run your program and move the camera around, the camera is directly behind the plane and does not provide a great view; the plane then speeds away from the camera, and when you move the camera, it is very easy to lose sight of the plane as

it is a small model that is less than 10 units long on a floor that is 2000 x 2000 units. You can solve these issues in the next few steps.

g. **Change your camera** from "**kFPS**" to "**kManual**". **Set its position** to (0, 40, -50) so that it is placed above and behind the plane. Then **rotate** it 40 units about the x-axis so that it is looking down at the plane.

```
myCamera->SetPosition(0, 40, -50);
myCamera->RotateX(20);
```

When you run your program, you will see that you have a better view of the plane when the program starts, but then the plane speeds away, and you lose sight of it. You can solve this by making the camera follow the plane while still keeping its initial distance and rotation relative to the plane.

h. Since the plane only moves forward (along the z axis) at this point, having the camera follow it means changing the z position of the camera as the z position of the plane changes, while keeping the other positions and rotations of the camera fixed. Therefore, below the code that moves the plane in the game loop, add a line of code that updates the z position of the camera to the z-position of the plane minus 50 (to maintain the same initial distance).

```
myCamera->SetZ(plane->GetZ() - 50);
```

Run your program, and you will see that the camera follows the plane as it moves forward.

i. Now, using part of the code you wrote to control a camera with mouse movements in the lab, **add code to the game loop** to **rotate** the plane along the y-axis **using mouse movement**.

```
int mouseMoveX = myEngine->GetMouseMovementX();
plane->RotateY(mouseMoveX * 0.1);
```

When running your program, you should note that although the plane rotates with mouse movements, it does not change its direction of movement. That is because it moved along the world-Z, not local-Z. Therefore, change the code that moves your plane to:

```
plane->MoveLocalZ( 0.05 );
```

j. The plane should be moving fine now, but you also need to update the camera so that it updates its x position when the plane is rotated and moves sideways. Therefore, below the code:

```
myCamera->SetZ(plane->GetZ() - 50);
```

Add the code:

```
myCamera->SetX(plane->GetX());
```

4. Shooting Balls

a. You now have a flying plane that you can control to some extent. Let us give the plane the ability to shoot out a ball when the space key is hit.

b. In the initialization section of your code, load the "**ball.x**" mesh, and declare a ball **IModel*** variable. **DO NOT CREATE THE BALL MODEL YET.**

c. In the game loop, write code to check if the space key is hit; if it is, then create a ball model at the plane's position, and assign it to the ball variable created earlier.

```
ball = ballMesh->CreateModel(
                  plane->GetLocalX(),
                  plane->GetLocalY(),
                  plane->GetLocalZ());
```

d. You should now see balls being created at the plane's origin every time you press the space key. However, the balls are too big, and they are not really "fired", they just sit there doing nothing.

e. You can **scale models** using the void **Scale(float scale)** method. You can use the method to scale the balls created by updating the code inside the if-condition that creates the ball model to the code below:

```
ball = ballMesh->CreateModel(
                   plane->GetLocalX(),
                   plane->GetLocalY(),
                   plane->GetLocalZ());
ball->Scale(0.05);
```

f. Next, **restrict the firing of balls to one at a time**. The fired ball gets automatically destroyed after travelling a certain distance, then another one can be created. First, create a **Boolean** variable called "**ballFired**" in the initialization section, and initialize it to **false**. Then, update the condition of the if statement that creates the ball model to check if the space key is hit and **ballFired** is **false** before creating the model. Then inside the if statement add code that changes **ballFired** to **true**.

```
ball= ballMesh->CreateModel(
```

```
                    plane->GetLocalX(),
                    plane->GetLocalY(),
                    plane->GetLocalZ());
ball->Scale(0.05);
ballFired = true
```

g. Next, create a float variable in the initialization section called "**ballDistance**" and initialize it to 0. Then, below the if-condition that creates the ball model in the game loop, add another if-condition that checks if **ballFired** is **true**; within it, add code to move the ball forward by 3 units and then add 3 to the variable **ballDistance**.

```
ball = ballMesh->CreateModel(
                    plane->GetLocalX(),
                    plane->GetLocalY(),
                    plane->GetLocalZ());
ball->Scale(0.05);
```

The **ballDistance** will be used to check how far a fired ball has travelled so that it can be destroyed after a certain distance. Inside the if-statement created above, add the if-statement below to destroy the ball when **ballDistance has exceeded 2000** (you can try out different numbers to see what works best for you), then set **ballFired** to false so that a new ball can be fired, and reset ballDistance to 0.

```
if (ballDistance >= 2000)
{
      ballMesh->RemoveModel(ball);
      ballFired = false;
      ballDistance = 0;
}
```

RemoveModel() method removes a model from a scene, but the pointer variable is still available to be reused. However, trying to manipulate the variable without assigning a model to it will cause an error and crash your program

h. There is still one issue left: the balls are not always fired in the direction the plane is facing. You can fix that by keeping a record of the units rotated by the plane, and rotating the ball when it is created. In the initialization section, declare and initialize a **float** variable named "**totalRotation**" to 0, and a **Boolean** variable named "**ballDirectionSet**" to false. Then, below the line of code that rotates the plane based on the movement of the mouse:

```
plane->RotateY(mouseMoveX * 0.1);
```

Add the code below to add the rotated units to **totalRotation**:

```
totalRotation += (mouseMoveX * 0.1);
```

Then, in the if-statement that checks if **ballFired** is true, add the code below:

```
if (firedDirectionSet == false)
{
        ball->RotateY(totalRotation);
        firedDirectionSet = true;
}
```

Finally, in the if-statement that checks if the ball has travelled enough distance to be destroyed, add a line of code to set **firedDirectionSet** to false after the ball is destroyed.

What's Next?

a. Use the **spike.x** mesh to create and position spikes all over your scene.

b. **A spike should disappear if it is hit by a ball fired by the plane**, but how do you know that a fired ball has hit a spike? Try different approaches.

c. Add **UI elements** in your **Field of Spikes** project to display the following information:

- The x, y, and z coordinates of the plane, listed as three separate lines and aligned to the left, at the bottom of your window.
- A blinking message asking the user to turn back if the user flies off the floor, positioned at the center of the window.
- Whether the player can shoot a ball or not, at the top left corner of the window.
- The total number of spikes at the start of the game and the number of spikes shot by the player (use format "Score: [shot]/[total] e.g. Score: 2/10), centered at the top of the window.